

DETECTION/ML HANDOFF

Ingestion -> Detection Integration Guide

Exact flow contract, earlier upstream flaws, raw-field requirements, and acceptance tests

Audience	Ingestion / PCAP / Zeek / parser team
Status	Integration handoff - do not redesign Detection
Baseline	Detection baseline: 254fc16 (verified)

Purpose This document defines the exact boundary between the finished Detection/ML module and your module. Follow it as a contract: preserve raw evidence, do not invent values, and do not move detection logic across the boundary.

SIH 2026 - AI-Based Detection of Cyber Threats in Unidirectional IP Traffic

1. Integration objective

Your job is to produce one truthful, typed flow record per line. Detection already owns rolling windows, scoring, model inference, alert aggregation, and ThreatAlert generation. The ingestion side should preserve observable network facts and raw metadata; it should not pre-decide what is malicious.

PCAP / live traffic passive one-way capture	Zeek conn / dns / ssl / http logs	Ingestion join by uid + emit JSONL	Detection adapter normalize to FlowEvent	7 detectors rules + DGA ML	ThreatAlert v1.1 POST / JSONL
---	---	--	--	--------------------------------------	---

Boundary rule Do not calculate detector-level rolling features upstream. Detection deliberately recomputes keyed event-time windows so PCAP replay and live traffic behave the same.

2. Required FlowEvent core contract

Each JSONL record must be able to supply these core fields. A malformed record is skipped rather than silently coerced. JSON numbers must be numbers, not quoted strings.

Field	JSON type	Requirement
timestamp	number	Finite epoch seconds. Prefer explicit top-level timestamp. If absent, current adapter can derive it from the final component of flow_id.
src_ip	string	Non-empty source/originator IP.
dst_ip	string	Non-empty destination/responder IP.
proto	string	Non-empty protocol, e.g. tcp/udp.
duration	number	Finite and ≥ 0 .
orig_bytes	integer	≥ 0 ; bytes sent originator -> responder.
resp_bytes	integer	≥ 0 ; reverse-direction bytes observed.
orig_pkts	integer	≥ 0 .
resp_pkts	integer	≥ 0 .

Strict typing "dst_port": "443" and "orig_bytes": "123" are malformed. Send 443 and 123 as JSON numbers. If an explicit timestamp is present but invalid, Detection rejects the record; it will not silently fall back.

3. Canonical JSONL record

Recommended shape. Optional blocks should appear only when that protocol metadata exists. Keep raw values as strings; derived fields may be supplied in addition, not instead.

```
{
  "flow_id": "10.0.0.20:10.0.0.53:53:udp:1725000000.125",
  "uid": "Cabc123",
  "timestamp": 1725000000.125,
  "src_ip": "10.0.0.20",
  "dst_ip": "10.0.0.53",
  "src_port": 51422,
  "dst_port": 53,
  "proto": "udp",
  "service": "dns",
  "duration": 0.012,
  "orig_bytes": 74,
  "resp_bytes": 190,
  "orig_pkts": 1,
  "resp_pkts": 1,
  "conn_state": "SF",
  "dns": {
    "uid": "Cabc123",
    "query": "kq3v9x2mzt7wp00.com",
    "qtype": "A",
    "rcode": "NOERROR",
    "query_length": 21,
    "query_entropy": 3.9,
    "subdomain_entropy": 4.1,
    "is_txt": false,
    "label_count": 2
  }
}
```

4. Fields that unlock each detector

Detector	Key / grouping	Minimum signal Detection needs	Important notes
Port scan	src_ip	dst_port, dst_ip, timestamp	Detection computes vertical/horizontal fanout itself. Do not send global unique-port counts as truth.
DDoS	dst_ip	src_ip, orig_pkts and/or orig_bytes, timestamp	Originator-side traffic drives qualification.
C2 beaconing	src,dst,dst_port,proto	accurate event timestamps	Detection computes inter-arrival timing per relationship.
DNS tunnelling	src_ip,dst_ip	DNS derived metadata: query_length, entropy, label_count, is_txt	Raw dns.query improves evidence but is not required for this rule detector.
Data exfiltration	src_ip,dst_ip	orig_bytes + timestamp	Outbound/originator bytes drive qualification; resp

Detector	Key / grouping	Minimum signal Detection needs	Important notes
			bytes are context only.
Encrypted malware	host pair / flow	tls.server_name or SNI derived features; JA3/JA3S/JA4 for signatures	No payload decryption. Exact raw fingerprint strings are required for IOC matching.
DGA domain	src_ip	raw dns.query	The 19 lexical features are computed from the domain characters. A numeric encoding cannot replace the domain string.

5. Critical raw fields - do not encode them away

Field	Priority	Why Detection needs the raw value
dns.query	CRITICAL	Required by the DGA model. It computes 19 lexical features from the exact domain text.
tls.server_name	CRITICAL	Used directly or to derive SNI length/entropy for encrypted-malware metadata heuristics.
tls.ja3	CRITICAL for signature path	Exact normalized string match against a trusted local IOC feed.
tls.ja3s	CRITICAL for signature path	Server-side TLS fingerprint; matched separately.
tls.ja4	CRITICAL for signature path	Matched separately; do not force it into JA3/MD5 representation.
dns.qtype / dns.rcode	Useful	Richer DNS evidence.
tls.version	Useful	Richer TLS evidence.
uid / src_port / service / raw conn_state	Useful	Correlation and analyst context.

Raw + derived is fine If another consumer needs an encoded ID, emit both the raw value and the encoding. Never replace dns.query, SNI, JA3/JA3S/JA4 with only an integer/category.

6. Earlier upstream flaws and how to avoid repeating them

Earlier flaw / risk	Why it was dangerous	Required correction / current rule
Global temporal/window features were not keyed	Port-scan state belongs to src_ip; DDoS breadth belongs to dst_ip; beacon timing belongs to a 4-tuple. Global values can mix	Do not make Detection depend on flow_rate, byte_rate, inter_arrival_mean/stddev, unique_dst_ports, unique_dst_ips or

Earlier flaw / risk	Why it was dangerous	Required correction / current rule
	unrelated flows.	src_ip_entropy. Detection recomputes keyed windows.
First-record flow_rate could explode (~1,000,000)	A zero/tiny denominator creates meaningless feature spikes.	Treat upstream rate fields as non-authoritative. Do not use them as detector inputs until independently corrected and validated.
Timestamp sorting/zipping could misalign features with flows	A correct number attached to the wrong flow is worse than a missing number.	Keep each flow and all of its metadata joined by uid/identity. Preserve event timestamps on the record; never zip independently sorted arrays.
Raw DNS/TLS strings were omitted	DGA and TLS fingerprint paths stayed dark even though derived numeric fields existed.	Emit raw dns.query, tls.server_name, tls.ja3, tls.ja3s, tls.ja4.
cipher_encoded placeholder based on string length	Length modulo 16 is not cryptographic identity or useful TLS evidence.	Do not invent numeric cipher/fingerprint surrogates. Preserve the real observable value if needed.
conn_state encoding omitted Zeek S0	S0 then collapses into unknown/0, losing a strong scan clue.	Emit raw conn_state. If keeping an encoding, add S0 explicitly and keep raw string too.
Repeated uid joins could retain only one row	Multiple protocol rows can be silently dropped/overwritten.	Define deterministic join semantics; preserve the relevant conn/dns/tls/http fields for the same uid and test duplicate/multiple rows.
Direction ambiguity	Exfil/DDoS logic depends on originator vs responder direction.	orig_* = Zeek originator/source -> responder/destination. resp_* = reverse. Do not swap based on packet volume.

7. Fields Detection deliberately ignores

Current Detection adapter recognizes but deliberately ignores these upstream global-window fields because they were previously unreliable or incorrectly scoped:

```
flow_rate byte_rate inter_arrival_mean inter_arrival_stddev
unique_dst_ports unique_dst_ips src_ip_entropy
```

Do not optimize for these fields Even if ingestion continues emitting them for another consumer, Detection will not trust them. The authoritative rolling state is inside Detection and keyed per detector.

8. How ingestion should feed the DGA model correctly

1. Capture the DNS transaction passively and preserve the queried domain as dns.query exactly as observed.
2. Join DNS metadata to the corresponding flow using Zeek uid or another deterministic identity. Do not normalize the domain into an integer/category.
3. Emit query, qtype, rcode and flow timestamp in the same JSONL record.
4. Detection normalizes the domain, computes its 19 lexical features, runs the trained Random Forest, applies the configured threshold (default 0.75), and performs source-level alert aggregation.

5. Ingestion must not compute dga_score, Threat Score, severity, or source aggregation. Those belong to Detection.

Model contract If dns.query is missing, DGA is correctly silent. Do not fabricate a placeholder domain just to make the detector run.

9. How ingestion should feed encrypted-malware detection

6. Preserve TLS handshake metadata only; do not decrypt payloads.
7. Emit tls.server_name when observable. Optionally also emit sni_length and sni_entropy, but never replace the raw SNI with only the derived values.
8. Emit raw JA3, JA3S and JA4 strings when available. Detection lowercases/normalizes and compares them against a trusted local feed.
9. Do not download threat feeds in ingestion and do not label UDP/443 as malicious. QUIC-specific fingerprint extraction is a future ingestion capability, not a current detector.

10. Acceptance checks before handing a branch to Detection

- [] At least one real/synthetic record has every required core field with correct JSON types.
- [] Explicit numeric timestamp survives unchanged; malformed timestamp is rejected, not silently rewritten.
- [] dns.query prints exactly from Detection adapter for a DNS record.
- [] tls.server_name, ja3, ja3s and ja4 print exactly for a TLS record.
- [] uid, src_port, service and raw conn_state are preserved when present.
- [] orig_bytes/orig_pkts represent source->destination; resp_* is reverse.
- [] No raw domain/fingerprint is replaced by an integer/category encoding.
- [] S0 is preserved in raw conn_state and is not collapsed to unknown.
- [] Multiple Zeek rows with the same uid are handled deterministically.
- [] Large/high-rate captures do not reorder metadata away from their owning flow.
- [] One JSON object is emitted per line; malformed records are surfaced, not silently patched.

11. Quick verification commands

From detection_core/ with the final ingestion features file available:

```
python -c "
from detection_core.adapters import IngestionJsonAdapter
for f in IngestionJsonAdapter(path='features.jsonl'):
    if f.dns and f.dns.query:
        print('dns.query:', f.dns.query)
    if f.tls and f.tls.server_name:
        print('tls.server_name:', f.tls.server_name)
    if f.tls and f.tls.ja3:
        print('tls.ja3:', f.tls.ja3)
"
```

Then run Detection locally:

```
python -m detection_core.runner features.jsonl --output alerts.jsonl
```

Expected handoff result Detection should not require a code change when these fields begin arriving. If the final ingestion contract changes names or nesting, notify Detection before merging rather than silently drifting the schema.

12. Definition of done for ingestion integration

- [] Final ingestion branch emits the critical raw fields.
- [] Current Detection adapter consumes them losslessly without patching ingestion inside detection branch.
- [] Realistic replay/capture includes scan, DDoS, beacon, DNS tunnel, exfil, DGA DNS queries and TLS metadata cases.
- [] Real benign capture is available for empirical false-positive measurement.
- [] DGA evaluation data has genuine family/seed labels if family-aware validation is required.
- [] No upstream feature is presented as ground truth unless its keying, timestamp alignment and direction semantics are tested.